neueda

More on Java Classes

Objectives

- Constructors
- ‘this’ reference
- Method overloading
- Static variables and methods

Constructors

- When you use the ‘new’ keyword
  - An object is created with the default values for all the instance variables ie. null, 0.0, 0, or false
- You can place in your code special methods called ***constructors***
- You can call constructors when you create your objects

Using Constructors

- To create an account with the appropriate name and balance, use the following

```
Account acc = new Account(“Nick”, 5000);
```

- For this to work, there *must be* a constructor in the Account class that takes in a String and a double

Defining Constructors

- Constructors are defined similarly to methods but
  - There is no return type
  - The name is always the class name

```
public class Account {
        private String name;
        private double balance;

        // constructor
        public Account (String s, double d) {
              name = s;
              balance = d;
        }
}
```

The Default Constructor

- When a class is defined, a default, no argument, public constructor is present
- This is called when the following code runs

```
Account acc = new Account();          // call to default constructor
```

- The default constructor ***is not*** present whenever you add a constructor

Syntax of Default Constructor

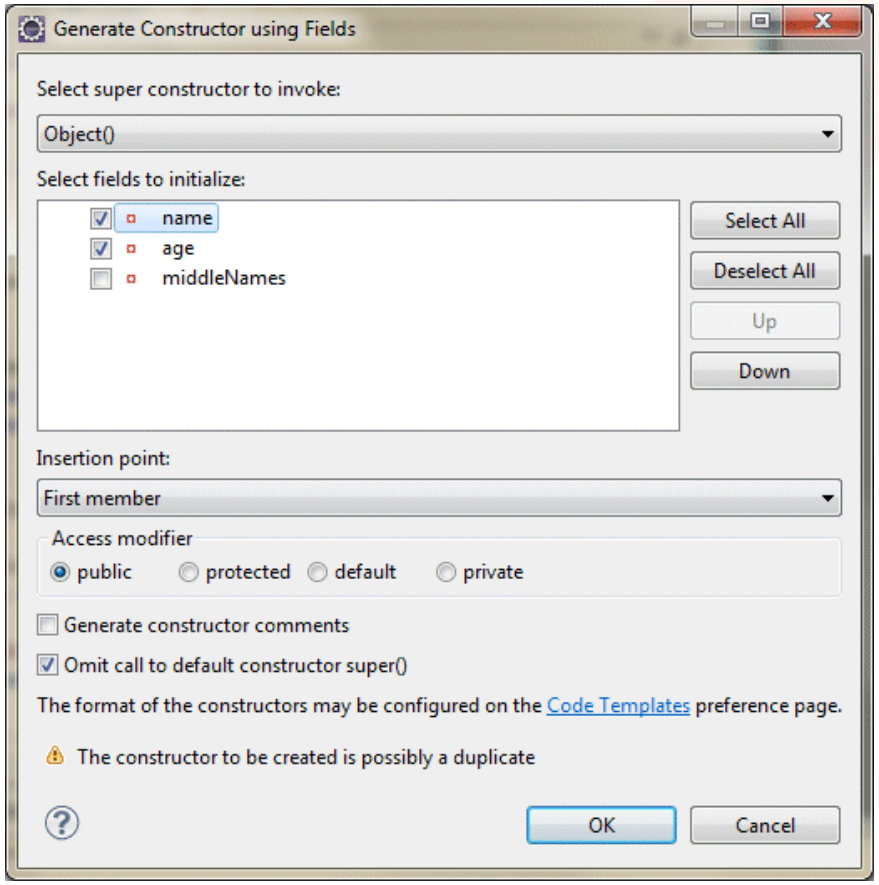
```
public class Account {
        private String name;
        private double balance;

        // this constructor is no longer present
        // unless I define it explicitly as below
        public Account ()  {
              // code can go in here if you wish, see next slide
        }

        // constructor
        public Account (String s, double d)  {
              name = s;
              balance = d;
        }
}
```

Generating Constructors from with an IDE

- Constructors can be generated using Eclipse or IntelliJ, in Eclipse
  1. Right click in the code
  2. Click **Source** and then click **Generate Constructor using Fields**



Constructors Calling Constructors

- One constructor can call another constructor using ***this***
- It must be the **first line** in the constructor

```
public class Account
{
      public Account (String s, double d)  {
            this(s);
            balance = d;
      }

      public Account(String n) {
            name = n;
      }
}
```

Instance Method Simplification using ‘this’

- Both instance variable and local method variable can have the same name
- To differentiate them we use this on the instance variable (*this object’s* balance variable)
  - **this**.balance = balance;

```
public class Account {
        private double balance;

        public void setBalance(double balance) {
                this.balance = balance;
        }

}
```

- ***this*** is a keyword that refers to the current object

Method Overloading

- Multiple methods with the **same** name
  - But take in *different* parameters

```
public class Account {
        public void withdraw() {
                withdraw(20);
        }

        public void withdraw(double d) {
                balance-=d;
        }
}
```

```
acc.withdraw();      // gives me £20
acc.withdraw(50);    // gives me £50
```

Static Methods and Variables

- Some variables and methods are not required for every instance
  - One shared copy is enough
  - If only one copy of a variable is required, use a ***static*** variable
- Static variable example
  - An interestRate variable in our Account class would have the same value for all the Account holders

Declaring Static Variables

- There is now **only one copy** of interestRate, irrespective of how many instances of Account there might be

```
public class Account {

        private double balance;
        private static double interestRate=0.2;

}
```

Using Static Variables

- Static variables can be used by all methods within a class

```
public class Account {
        private double balance;
        private static double interestRate=0.2;

        public void addInterest() {
              balance = balance +  (balance*interestRate);
        }

}
```

Initialising Static Variables

- Instance variables can be initialised in a constructor, but what about static variables?
  - Static variables can be initialised in a **static initialiser** block

```
public class Account {
        private double balance;
        private static double interestRate;

        // here is the static initialiser. It will run when the class loads
        static  {
            interestRate = 1.2 ; // or get some value from a DB or elsewhere
        }
}
```

Static Methods

- Static methods can be used to manipulate static variables
- Static methods are called on the class
  - ClassName.staticMethodName()

```
public class Account {
        private static double interestRate=0.2;
        public static double getInterestRate() {
                return interestRate;
        }
}
```

```
double rate = Account.getInterestRate();
```

The Math Class

- The Math class in Java has a number of static methods such as
  - Math.sqrt(), Math.random(), Math.tan()

```
double someRandomNumber = Math.random() * Math.PI;
```

- Math also has some static final properties as well
  - **Math.PI**
  - **Math.E**

Static Imports

- If you wanted to use the Math class constants and static methods a great deal in a class

```
double result = Math.PI * Math.random() + Math.tan(20);
```

- Using a static import allows you to miss off the class name on the static methods and properties

```
import static java.lang.Math.*;
public class TestMaths {
  public static void main(String[] args) {
       double result = PI * random() + tan(20);
  }
}
```

The main Method

- The main method is where your program starts
  - **public** – the method is visible from outside the class
  - **static** - there does not need an instance of the class to run
  - **void** - main has a return type of void as it does not return anything
  - **String[] args** - main takes in an array of Strings from the command line

```
public static void main(String[] args)
```

Summary

- Constructors
- ‘this’ reference
- Method overloading
- Static variables and methods